Razorpay AI Buildathon 2026: The Complete Application Guide

75K/Month Stipend. Build-First Selection.
No Resume Screen. 5 Tracks Decoded.
What to Build. How to Submit. How to Win.

By **OneRoadmap** | oneroadmap.io
Follow @**thegauravghai** for daily career updates

Students Only | 6 or 12 Months | In-Person Bangalore | From September 2026

What's Inside

01. How Selection Actually Works (The Build-First Model)

Most internship hiring starts with a resume screen, an aptitude test, and a group discussion that has nothing to do with the actual job. Razorpay's AI Buildathon throws that playbook out entirely. There is no resume screen. No aptitude test. No group discussion. No CGPA cutoff. No college tier filtering. The only thing that matters is what you build.

The process has exactly four steps. One, you pick a track. Two, you build a working AI project. Three, you submit a public GitHub repo, a 5-minute pitch video, and architecture documentation. Four, if your build has signal, Razorpay calls you in for a panel review. That's it. Your college name doesn't appear in the evaluation. Your CGPA is irrelevant. Your resume doesn't exist in this process.

The 4 Evaluation Signals (From Razorpay's Official Criteria)

Signal	What It Means	What Proves It
Problem Taste	Did you pick a meaningful, real-world financial or merchant problem? Not a toy demo.	Your project solves something a real business would pay for, not a class assignment.
Build Quality	Is the code clean? Does it actually run? Can someone else set it up from your README?	Clean repo structure, execution reliability, clear setup instructions, working demo.
AI Judgment	Did you use AI where it genuinely adds value? Or did you force-fit a model where a simple if-else would work?	You chose AI for the right parts AND used deterministic logic where AI wasn't needed.
Failure Recovery	What broke during development? How did you fix it? What limitations remain?	You show what failed, how you debugged it, and what's still unsolved. No pretending everything works perfectly.

WHAT PANELS LOOK FOR: The panel isn't looking for a perfect system. They're looking for an engineer who thinks clearly about real problems, builds working solutions, uses AI appropriately (not everywhere), and is honest about what doesn't work yet. A project with 78% accuracy and an honest exception list beats a project claiming 99% accuracy with a cherry-picked demo.

02. The Five Tracks, Decoded

Track 1: AI Growth & Agentic Commerce

The real problem: Make merchants sell more and make them transactable by AI buyers end-to-end.

What Razorpay is solving: NPCI's Unified Agent Protocol (UAP) and global protocol races (ACP, AP2, x402) are making agent-to-agent commerce the open problem of 2026. Razorpay's in-app commerce pilots are already live. They need builders who can make checkout conversational, catalogs AI-readable, and upsell workflows automated.

Project ideas you could build: (1) A conversational checkout agent that completes purchases via natural language using Razorpay test-mode APIs, handling payment, address, and confirmation in a chat flow. (2) An AI-readable product catalog system that converts unstructured merchant product data into structured, agent-queryable format. (3) An upsell and cross-sell agent that analyzes purchase history and recommends complementary products at checkout, measuring conversion uplift.

Best for: Full-stack developers, LLM enthusiasts, anyone interested in conversational AI and commerce. Need comfort with APIs, payment flows, and agent design patterns.

> **FROM THE OFFICIAL PAGE:** The bar: 'Every money action explainable, bounded and gated. Show the audit trail and one failure handled gracefully.' Translation: your agent must log every transaction decision, have spending limits, and demonstrate what happens when something goes wrong.

Track 2: AI Risk Manager

The real problem: Stop merchants losing money to fraud, returns, and chargebacks.

What Razorpay is solving: AI-enabled fraud is hitting Indian BFSI hard. Returns and chargebacks quietly eat merchant margins. Traditional rule-based systems can't keep up. Razorpay needs risk-minded builders who can detect and respond to financial threats in real-time.

Project ideas: (1) A chargeback evidence responder that automatically gathers transaction evidence, compiles it into the card network's dispute format, and generates a response brief. (2) A return-risk scorer that predicts which orders are likely to be returned based on buyer behavior, product category, and historical patterns. (3) A fraud-spike detector that monitors transaction streams for anomalous patterns and alerts with root cause analysis.

Best for: ML/data science students, anyone who loves classification problems, precision-recall tradeoffs, and anomaly detection. Python + scikit-learn + pandas comfort required.

> **FROM THE OFFICIAL PAGE:** The bar: 'Honest metrics including false-positive cost. Strictly defense-only: anything offense-capable is disqualified.' Translation: you MUST report precision, recall, and the business cost of false positives. And don't build anything that could be used to commit fraud.

Track 3: AI Revenue Recovery

The real problem: Find revenue that's slipping away and win it back.

What Razorpay is solving: Revenue loss rarely happens in one clean step. A payment degrades, a checkout gets abandoned, a subscription fails, or an invoice goes overdue. AI can now close the loop from detecting the problem to diagnosing it, choosing the right intervention, and actually recovering the money.

Project ideas: (1) A payment failure recovery agent that detects declined payments, identifies the root cause (card limit, bank error, network issue), and suggests or executes the optimal retry strategy. (2) A checkout abandonment recovery system that identifies drop-off points, triggers personalized recovery messages (email, SMS, WhatsApp), and tracks recovery rates. (3) A B2B receivables chaser that monitors overdue invoices and generates escalating collection workflows: gentle reminder, firm follow-up, escalation notice.

Best for: Workflow automation builders, anyone who thinks in state machines and decision trees. Need comfort with agentic loops: detect, diagnose, decide, execute, verify.

> **FROM THE OFFICIAL PAGE:** The bar: 'Show measured money recovered across a batch, with compliant escalation, stopping rules, and an audit trail.' Translation: run your agent against a batch of test cases and show exactly how much revenue it recovered, when it stopped pushing, and the complete decision log.

Track 4: AI Finance Controller

The real problem: Run the books and the cash position automatically.

What Razorpay is solving: The 2026 builder consensus: verification capacity, not generation speed, is the bottleneck. Reconciliation, settlement, and forecasting are still done by hand at most companies. This track is about automating the finance back-office.

Project ideas: (1) A multi-source reconciliation agent that matches transactions across bank statements, payment gateway reports, and accounting software, flagging discrepancies. (2) A settlement Q&A; agent where finance teams ask natural language questions about settlement reports and get accurate, sourced answers. (3) A forward cash forecaster that predicts cash position 7-30 days ahead based on receivables, payables, and seasonal patterns.

Best for: Data engineering and analytics students, anyone comfortable with messy tabular data, matching algorithms, and financial concepts (doesn't need to be a CA, just willing to learn).

> **FROM THE OFFICIAL PAGE:** The bar: 'Throughput plus measured accuracy plus an honest exception list. One cherry-picked match proves nothing.' Translation: run your agent on 50+ records of synthetic data. Report how many it matched correctly, how fast, and which ones it couldn't resolve.

Track 5: Open Track

The real problem: Whatever problem YOU think should be solved.

What Razorpay is saying: The best ideas don't always fit predefined categories. This track exists for builders who see an opportunity Razorpay didn't. Any domain, workflow, or user is fair game.

Project ideas: (1) An AI-powered merchant onboarding assistant that reads KYC documents, extracts entity data, verifies against public databases, and auto-fills onboarding forms. (2) A developer documentation chatbot that answers API integration questions using Razorpay's docs as context (RAG application). (3) An AI disputes mediator that helps merchants and buyers resolve transaction disputes through structured dialogue.

Best for: Independent thinkers with a strong problem statement. The open track is NOT easier. The bar is the same. You just get to choose the problem.

> **FROM THE OFFICIAL PAGE:** Open doesn't mean lower bar. 'Show a real problem, a working product, meaningful use of AI, and evidence that it creates value. The same bar for execution, reliability, and depth applies here.'

03. Which Track Should You Pick (Decision Framework)

Your Strength	Best Track	Why
I'm a frontend/full-stack dev who likes building user-facing products	Track 1: AI Growth & Agentic Commerce	Heavy on conversational UI, API integration, and checkout flows. Less ML, more product.
I love ML, classification, precision-recall, and anomaly detection	Track 2: AI Risk Manager	Pure ML track. Model training, evaluation metrics, feature engineering.
I think in workflows: detect, decide, act, verify loops	Track 3: AI Revenue Recovery	Agentic workflows with state machines. Less model training, more orchestration.
I'm comfortable with messy data, matching algorithms, and tabular processing	Track 4: AI Finance Controller	Data engineering + matching logic. Think reconciliation, not neural networks.
I have a specific problem I'm obsessed with solving	Track 5: Open Track	Only if your problem is STRONG. A weak open-track idea will lose to a strong Track 1-4 submission.
I can't decide	Track 3 or Track 4	These have the most concrete, measurable outcomes. Easiest to demonstrate 'working + measured + honest.'

PRO TIP: Don't pick the track that sounds most impressive. Pick the one where you can ship a working system with measured results before the deadline. A working checkout recovery agent with 65% recovery rate beats a half-built fraud detection system with 'impressive ML architecture' that doesn't actually run.

04. Building a Project That Gets Shortlisted

What 'Working + Measured + Honest' Looks Like

Working: Someone can clone your repo, follow your README, and have the system running in under 15 minutes. Not 'it works on my machine.' It works on ANY machine that follows your setup instructions. If you're using APIs, provide test-mode keys or mocks. If you need a database, include a seed script. The system must do something real when launched, not just print 'Hello World.'

Measured: You've run your system against a meaningful test set and reported the results honestly. For Track 2: precision, recall, F1 score on a held-out test set. For Track 3: dollars recovered across a batch, recovery rate percentage. For Track 4: match rate across 50+ records, throughput per second, exception count. Vague claims like 'the system works well' mean nothing. Numbers mean everything.

Honest: You explicitly list what your system can't do, where it fails, and what you'd fix with more time. Razorpay's official page says 'one cherry-picked match proves nothing.' They want to see that you tested broadly, not just on the one example that happens to work perfectly.

Common Mistakes That Kill Submissions

X

Cherry-picked demos

Running your fraud detector on 5 hand-picked transactions that all classify correctly. The panel will ask: 'What happens on 500 random transactions?' If you don't know, you've already lost.

X

AI everywhere syndrome

Using GPT-4 to do something a regex or SQL query handles perfectly. Razorpay explicitly evaluates 'AI Judgment': using AI where it adds value AND choosing deterministic solutions where AI is unnecessary. Show you know the difference.

X

No failure documentation

Submitting a project as if nothing ever broke. Real engineering involves failures. Document them: 'The initial reconciliation approach using exact string matching failed on 40% of records because merchant names had inconsistent formatting. I switched to fuzzy matching with Levenshtein distance, improving match rate to 84%.' That sentence shows more engineering maturity than a flawless demo.

X

Unrunnable repos

A README that says 'install dependencies and run.' Which dependencies? What Python version? What environment variables? If the panel can't run your project in 10 minutes, they move on.

Scope Advice: What's Buildable Solo Before the Deadline

Scope DOWN, not up. A narrow, working system beats an ambitious, half-built one every time. Examples of good scoping: 'Payment failure recovery agent for UPI failures only (not all payment methods).' 'Chargeback evidence compiler for one card network (Visa only).' 'Reconciliation agent for bank statement + Razorpay settlement report (two sources, not five).' The panel will not penalize you

for narrow scope. They will penalize you for broken ambitious scope.

05. The Submission, Field by Field

The application form is a Google Form at razorpay.com/buildathon. Here's how to answer each field well.

Project Name / Title

Be specific and descriptive. Not: 'AI Agent' or 'Fraud Detector.' Instead: 'UPI Payment Failure Recovery Agent' or 'Multi-Source Settlement Reconciliation Engine.' The title should tell the reviewer exactly what your project does in under 10 words.

Project Objectives (What It Solves)

2-3 sentences maximum. State the problem, your approach, and the measurable outcome. Example: 'Merchants lose 3-5% of revenue to failed UPI payments that could be recovered with timely retries. This agent monitors payment failures, classifies root causes, and executes optimal retry strategies. Tested on 200 synthetic transactions with 67% recovery rate.'

GitHub Repository

Your repo MUST contain:

(1) Clean README with: project description, architecture diagram (even a simple one), setup instructions (step by step), demo instructions (how to see it working), known limitations. (2) requirements.txt or package.json with pinned versions. (3) .env.example file showing required environment variables (without actual keys). (4) src/ folder with organized code (not one 2000-line file). (5) tests/ folder with at least basic tests. (6) data/ folder with sample or synthetic test data. (7) docs/ folder with architecture notes.

5-Minute Pitch Video

This is covered in detail in Chapter 6.

Build Challenges & Technical Obstacles

This is where most applicants give fluff ('It was challenging but rewarding'). Instead, be specific and technical. Example: 'Initial approach used exact string matching for transaction reconciliation. Failed on 40% of records due to inconsistent merchant naming conventions. Switched to TF-IDF vectorization with cosine similarity, achieving 84% match rate. Remaining 16% require manual review due to completely missing metadata. Next step: add OCR for scanned bank statements.' This answer shows: real engineering thinking, honest failure admission, specific technical solution, measured improvement, and clear next steps.

06. The 5-Minute Pitch Video: Exact Script Structure

Your pitch video is 5 minutes. Not 4. Not 7. Exactly 5. Here's the second-by-second breakdown:

Time	Section	What to Say	Screen to Show
0:00-0:30	The Problem (30 sec)	'Merchants on Razorpay lose X% of revenue to [specific problem]. Currently this is handled [manually / not at all]. I built [Project Name] to solve this.'	A slide with the problem statement and one data point showing the scale.
0:30-2:30	Live Demo (2 min)	Walk through your system actually working. Not slides. Not code. The actual running system processing real-ish data. Narrate what's happening: 'Here I'm feeding 50 transactions. The agent is classifying each failure type. Watch: it identifies this as a bank timeout and triggers a retry...'	Screen recording of your system running. Terminal output, dashboard, or UI showing real processing.
2:30-3:30	Architecture (1 min)	'Here's how the system is built. [Component A] handles [function]. [Component B] communicates with [Component A] via [protocol]. I chose [technology] because [specific reason]. Data flows from [source] through [pipeline] to [output].'	Architecture diagram (can be a simple hand-drawn or Excalidraw diagram). Show the components and data flow clearly.
3:30-4:30	Results + Accuracy (1 min)	'I tested on [N records]. Match rate: [X%]. Precision: [Y%]. Recall: [Z%]. Recovery rate: [W%]. Processing time: [N sec per record]. The main failure mode is [specific]. I'd fix this by [specific approach].'	A results table or chart. Show the numbers. Not words. Numbers.
4:30-5:00	Limitations + Next Steps (30 sec)	'Three things I'd improve: [1], [2], [3]. The biggest unsolved challenge is [specific]. Given more time, I'd [specific next step].' End with: 'Thank you. Repo link in the description.'	A slide listing 3 limitations and 3 next steps clearly.

Video Production Tips

Record with OBS Studio (free) or Loom (free tier). Use a clean desktop. Close all unnecessary tabs. Test your audio before recording. Speak clearly and at a moderate pace. Don't read from a script word-for-word but have bullet points visible. Edit out any dead air or screen loading delays. Export at 1080p. Upload to YouTube (unlisted) or Google Drive and include the link in your submission.

PRO TIP: The demo section (0:30-2:30) is the most important 2 minutes of your entire application. If your system visibly works on screen, processing real data and producing meaningful output, you're ahead of 80% of applicants who show slides about what their system theoretically does.

07. Panel Round Prep: Defending Your Build

If your submission has signal, Razorpay invites you to a panel review. This is not a traditional interview with DSA questions. It's a build defense. The panel will have seen your repo, watched your video, and come with specific questions about your project.

Questions the Panel Will Ask (Be Ready)

1

'Walk me through your architecture decisions'

They want to hear WHY you chose each component. 'I used Redis for caching because the reconciliation agent makes repeated lookups on the same merchant data, and Redis gives me sub-millisecond reads.' Not: 'I used Redis because it's popular.'

2

'What happens when [component X] fails?'

They'll pick a component and ask about failure modes. 'If the LLM API times out mid-reconciliation, the agent saves the batch state to disk and retries from the last checkpoint. No matched records are lost.' Have failure scenarios prepared for every major component.

3

'Why AI here and not a rule-based approach?'

This tests your AI Judgment. 'I used AI for classifying payment failure root causes because the error messages from different banks are inconsistent and unstructured. A rule-based approach would need 200+ rules for bank-specific error codes. The LLM handles novel error messages by reasoning about the text. However, for the retry logic itself, I used deterministic rules because retry timing must be predictable and auditable.'

4

'Show me a case where your system fails'

They want honesty. Pull up a specific failure case from your test results. 'Here's a transaction where the agent matched incorrectly because both the merchant name and amount were identical for two different transactions on the same day. My current approach can't distinguish these without a unique transaction ID. I'd fix this by adding timestamp-based disambiguation.'

5

'How would you scale this to 10x volume?'

They want to see if you've thought beyond the demo. 'Currently processing 50 records takes 45 seconds. At 500 records, the bottleneck would be the LLM API calls. I'd batch similar transactions together to reduce API calls by 60%, add async processing with a task queue, and cache common pattern matches.'

6

'What would you build next if you had 6 months?'

This is your chance to show vision. Connect your current project to Razorpay's broader ecosystem. 'I'd extend the reconciliation agent to handle three-way matching (bank + Razorpay + accounting software), add a natural language interface for finance teams to query exceptions, and build an auto-learning system that improves match rules from manually resolved exceptions.'

08. Deadline, Application Link & Scam Warning

How to Apply

1 **Go to the official page**

razorpay.com/buildathon. Read the full page. Understand the tracks and the bar for each.

2 **Pick your track**

Use the decision framework from Chapter 3. Decide within 24 hours. Don't overthink.

3 **Build your project**

This is the hardest step. Allocate 2-3 weeks of focused building time. Scope down aggressively.

4 **Prepare your submission**

Clean your repo. Record your 5-min pitch video. Write your architecture docs. Fill the form.

5 **Submit via the Google Form**

The official application link: forms.gle/d9r2gvxp8cmoZhon9 (linked from razorpay.com/buildathon). Fill every field carefully.

Deadline

Some sources indicate September 5, 2026 as the deadline, while Razorpay's official page does not list a fixed closing date as of this writing. The internship starts in September, which means shortlisting and panels will move quickly once submissions pick up. Treat this as rolling selection. Do NOT wait until the last day. Submit as early as possible. Verify the current deadline on razorpay.com/buildathon before planning your timeline.

SCAM WARNING (From Razorpay)

Razorpay only contacts candidates through official Razorpay email addresses ending with @razorpay.com or @f5.com, or via auto-notifications from Workday. If you receive communication from any other email address claiming to be Razorpay or the Buildathon, it is likely a scam. Do NOT share personal information, pay any fee, or click suspicious links. Razorpay never charges candidates for hiring. Report suspicious messages to Razorpay directly.

OneRoadmap: Your Career Launchpad

Free certifications, AI career tools, 600K+ community.
Build your resume: theperfectresume.ai
Main site: oneroadmap.io

Follow @thegauravghai on Instagram
Daily AI tools, internship alerts, career hacks. 600K+ community.
instagram.com/thegauravghai

No resume screen. No aptitude test. No CGPA filter.
Just you, a real problem, and a working build.
This is the most meritocratic internship in India right now.
Pick your track. Start building. Submit.

Disclaimer: This guide is for educational purposes only. All information is sourced from razorpay.com/buildathon and public sources. Verify all details on the official page before applying. Not affiliated with Razorpay.

Made with intention by the OneRoadmap team. Share this guide freely.